

4. 文档与视图



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南开大学

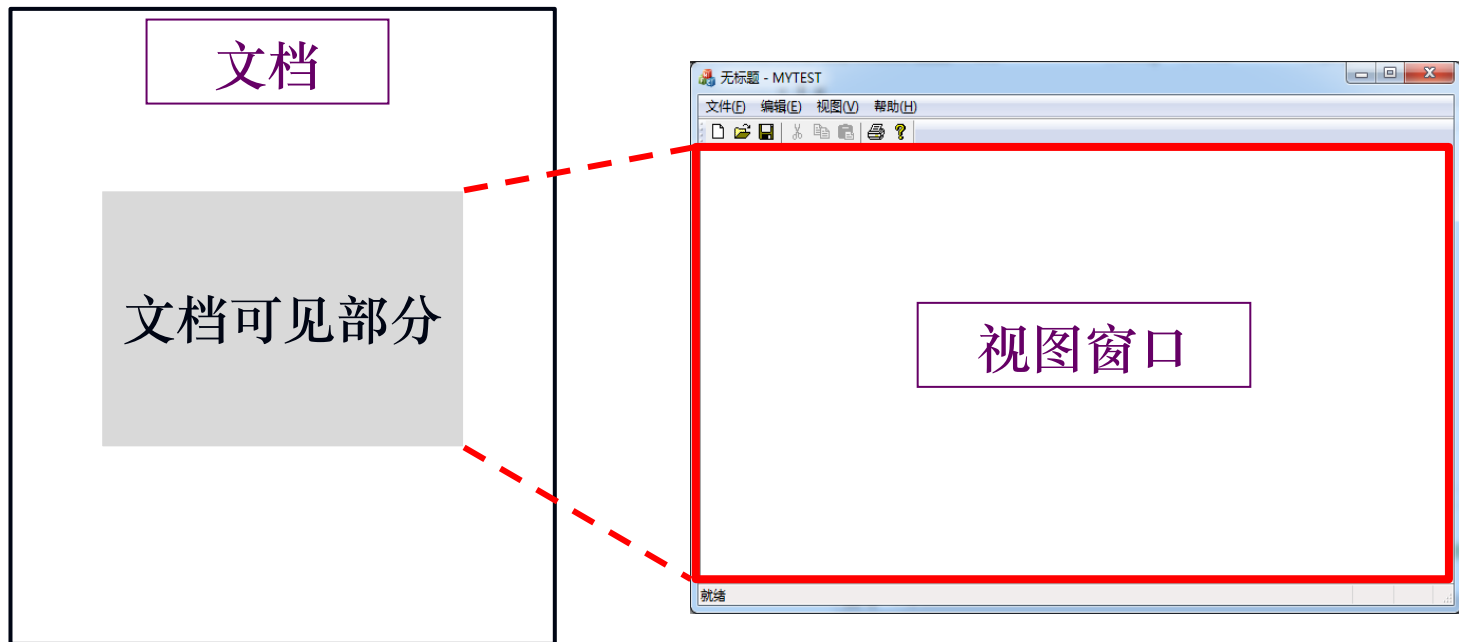
Nankai University

主要内容

- 文档与视图
 - 资源的应用
 - WM_TIMER消息
 - 综合实例
-
- 对应课本：第十、十一章

文档与视图

- 为了统一和简化数据处理方法，MFC中使用文档/视图结构
- 数据处理分为数据的管理和显示
 - 文档负责数据管理，包括数据的读取、存储和修改
 - 视图负责与用户交互，将文档类数据以某种方式显示



文档类

- 文档是MFC应用程序中所处理数据的集合，包括文本、图形、图像和表格数据
- 文档类的基类是**CDocument**，主要作用是把对数据的处理从对用户界面的处理中分离出来，集中处理数据，同时提供了与其它类交互的接口
- 文档负责数据读写操作，**数据通常保存在文档类的成员变量中**
- 文档类通过一个称为串行化的成员函数**Serialize()**将成员变量的数据保存到磁盘文件中，MFC应用程序为数据的串行化提供了默认支持

文档类



函数类型	说明	函数类型	说明
operator<<	写字节流	operator>>	读字节流
IsStoring	写状态	IsLoading	读状态
WriteString	写字符串	ReadString	读字符串

系统程序

```
void CMYTESTDoc::Serialize(CArchive& ar)
{
    if(ar.IsStoring())    //写操作, (insertion, 插入)
        ar<<...;
    else                  //读操作, (extraction, 取出)
        ar>>...;
}
```

视图类

- 视图是文档在屏幕上的映像，用户通过视图看到文档，并通过视图改变文档，视图是文档与用户间的中介
- 视图类的基类是**CView**，视图类直接或间接访问文档类的成员变量，显示文档中的数据，并把用户的输入解释为对文档的操作
- 每个文档都有一个或多个视图，但一个视图只能与一个文档对象相关联
- 用户通过与文档相关联的视图与文档进行交互，当用户打开一个文档时，应用程序就会创建一个与之相关联的视图
- 视图可以输出到窗口或打印机

视图类

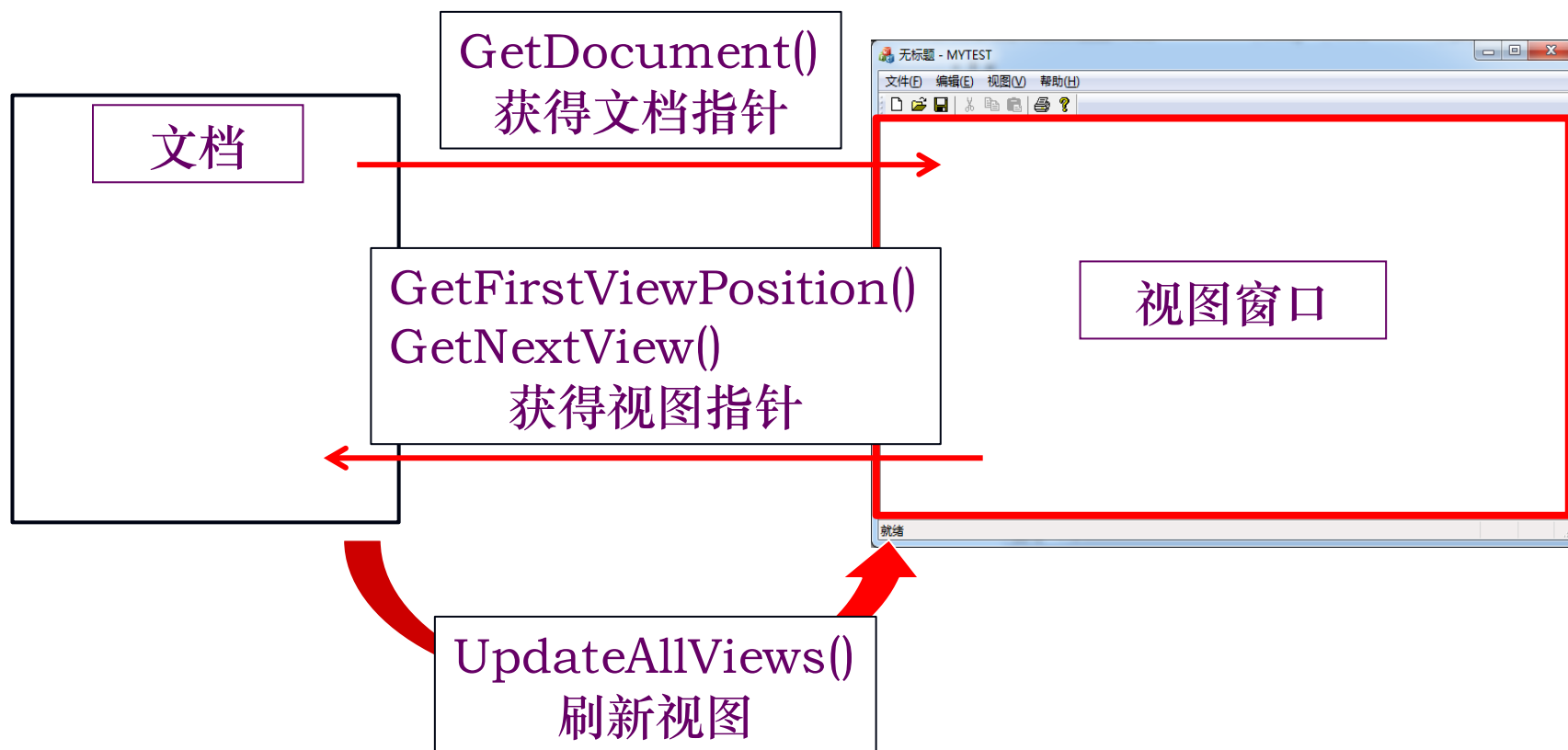
➤ 绘图：OnDraw()函数

- 在屏幕发生变化或焦点变化需要重绘时，调用OnDraw()
- 使用该函数保证程序切换后屏幕的正确显示
- **注意：尽量不要在OnDraw()函数之外调用绘图方法！**
- 通过视图或文档维护数据模型，在其他地方更改数据模型，在OnDraw()函数中根据该模型绘制

➤ 视图重绘

- Invalidate(): 该函数使整个窗口客户区无效，Windows会在应用程序的消息队列中放置WM_PAINT消息
- UpdateWindow(): 直接发送一个WM_PAINT消息，使窗口立即重绘

文档与视图的联系





视图类的成员函数GetDocument()

- 一个视图对象只有一个与之相关联的文档对象
- 在MFC应用程序中，视图对象通过调用GetDocument()得到与视图相关联的文档对象的指针，利用这个指针可以访问文档类及其派生类的公有成员

系统程序

```
#ifndef _DEBUG // MYTESTView.cpp 中的调试版本
inline CMYTESTDoc* CMYTESTView::GetDocument() const
{ return reinterpret_cast<CMYTESTDoc*>(m_pDocument); }
#endif
```

文档类的成员函数UpdateAllViews()

- 当文档的数据发生改变时，必须通知与它关联的每一个视图，以便更新所显示的数据
- 更新与文档有关的所有视图的方法是调用成员函数

```
void UpdateAllViews(CView* pSender,  
    LPARAM lHint = 0L, CObject* pHint=NULL );
```

- 如果在文档派生类的成员函数中调用该函数，第一个参数pSender设为NULL，表示所有与当前文档相关的视图都要重绘
- 如果在视图派生类的成员函数中通过当前文档的指针调该函数，第一个参数pSender设为当前视图，形式如下：

```
GetDocument()->UpdateAllViews(this)
```

视图类的成员函数OnUpdate()

- 当程序调用CDocument::UpdateAllViews()函数时，实际上调用了所有相关视图的OnUpdate()函数，从而更新相关的视图，可直接在视图派生类中调用该函数刷新当前视图
- OnUpdate()通过调用函数CWnd::Invalidate()刷新整个客户区，可以在自己的CWnd派生类中直接调用该函数

➤ **总结：**刷新视图时默认的函数调用过程：

CDocument::UpdateAllViews()
→ CView::OnUpdate()
→ CWnd::Invalidate()
→ OnPaint()
→ OnDraw()

编程工作



1. 在**文档类**中加入数据成员及相应读写函数
2. 按照需求添加命令处理函数
3. 在**视图类**中编写成员函数用于实现文档的显示和修改
4. 在**文档类**实现数据成员的保存

举例：文档和视图的交互

- 单击鼠标左键，在视图中显示点击位置，并进行保存



举例：文档和视图的交互

➤ 1、在文档类中加入数据成员及相应读写函数

在CMYTESTDoc类定义中(MYTESTDoc.h)

private:

CString m_str; // 显示的字符串

CPoint m_point; // 鼠标点击位置

public:

void Set(CPoint point,CString string);

void Get(CPoint &point,CString &string);

MYTESTDoc.cpp

CMYTESTDoc::CMYTESTDoc(){

 m_str.Empty();

 m_point = CPoint(0,0);

}

添加蓝色
代码

举例：文档和视图的交互

➤ 1、在文档类中加入数据成员及相应读写函数

MYTESTDoc.cpp

```
void CMYTESTDoc::Set(CPoint point,CString string)
{
    m_point=point;
    m_str=string;
}
```

```
void CMYTESTDoc::Get(CPoint &point,CString &string)
{
    point=m_point;
    string=m_str;
}
```

举例：文档和视图的交互

➤ 2、按照需求添加命令处理函数（响应鼠标左键单击消息）

MYTESTView.cpp

```
void CMYTESTView::OnLButtonDown(UINT nFlags, CPoint point)
{
    CMYTESTDoc* pDoc=GetDocument();
    CString str;
    str.Format(_T("鼠标单击位置: (%d,%d)"), point.x,point.y);
    pDoc->Set(point,str);
    Invalidate();
    CView::OnLButtonDown(nFlags, point);
}
```

添加蓝色
代码

举例：文档和视图的交互

➤ 3、在视图类中编写成员函数用于实现文档的显示和修改

MYTESTView.cpp

```
void CMYTESTView::OnDraw(CDC* pDC)
{
    CMYTESTDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);
    if (!pDoc)
        return;
    // TODO: 在此处为本机数据添加绘制代码
    CPoint pt;
    CString str;
    pDoc->Get(pt,str);
    pDC->TextOut(pt.x,pt.y,str);
}
```

添加蓝色
代码

举例：文档和视图的交互

➤ 4、在文档类实现数据成员的保存

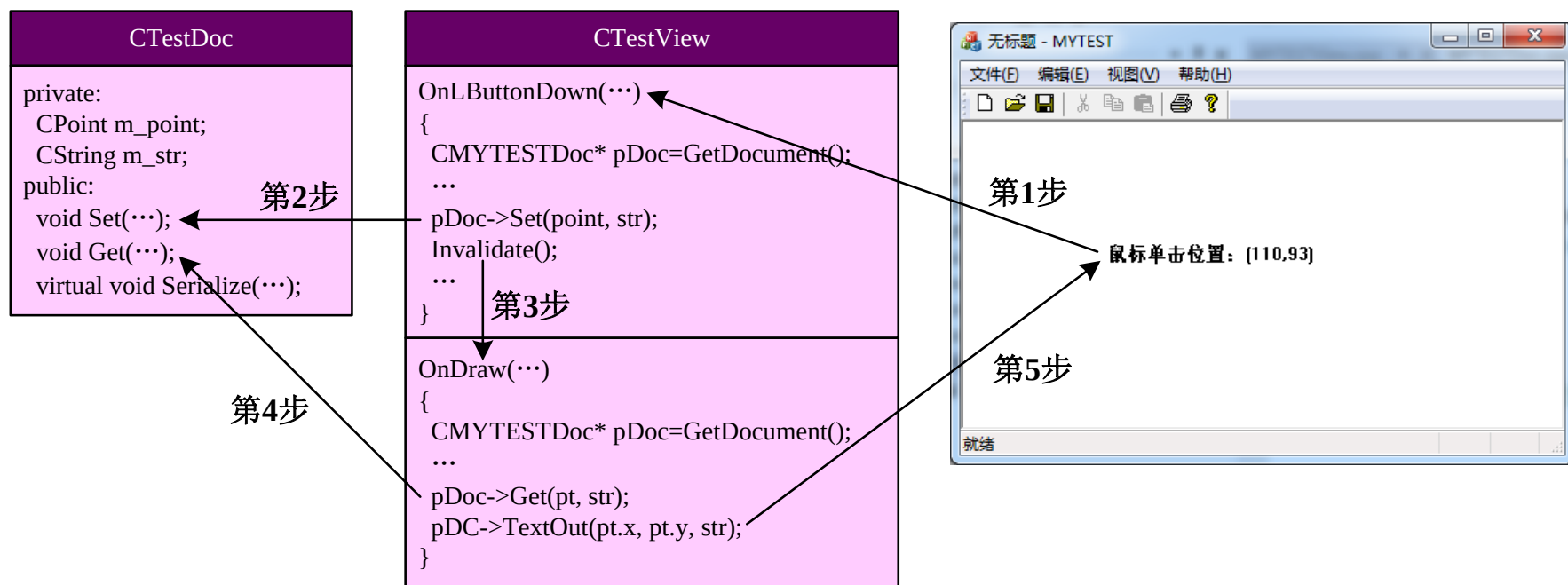
MYTESTDoc.cpp 修改Serialize()函数

```
void CMYTESTDoc::Serialize(CArchive& ar)
{
    if(ar.IsStoring()) //写操作
        ar<<m_point<<m_str;
    else                //读操作
        ar>>m_point>>m_str;
}
```

添加蓝色
代码

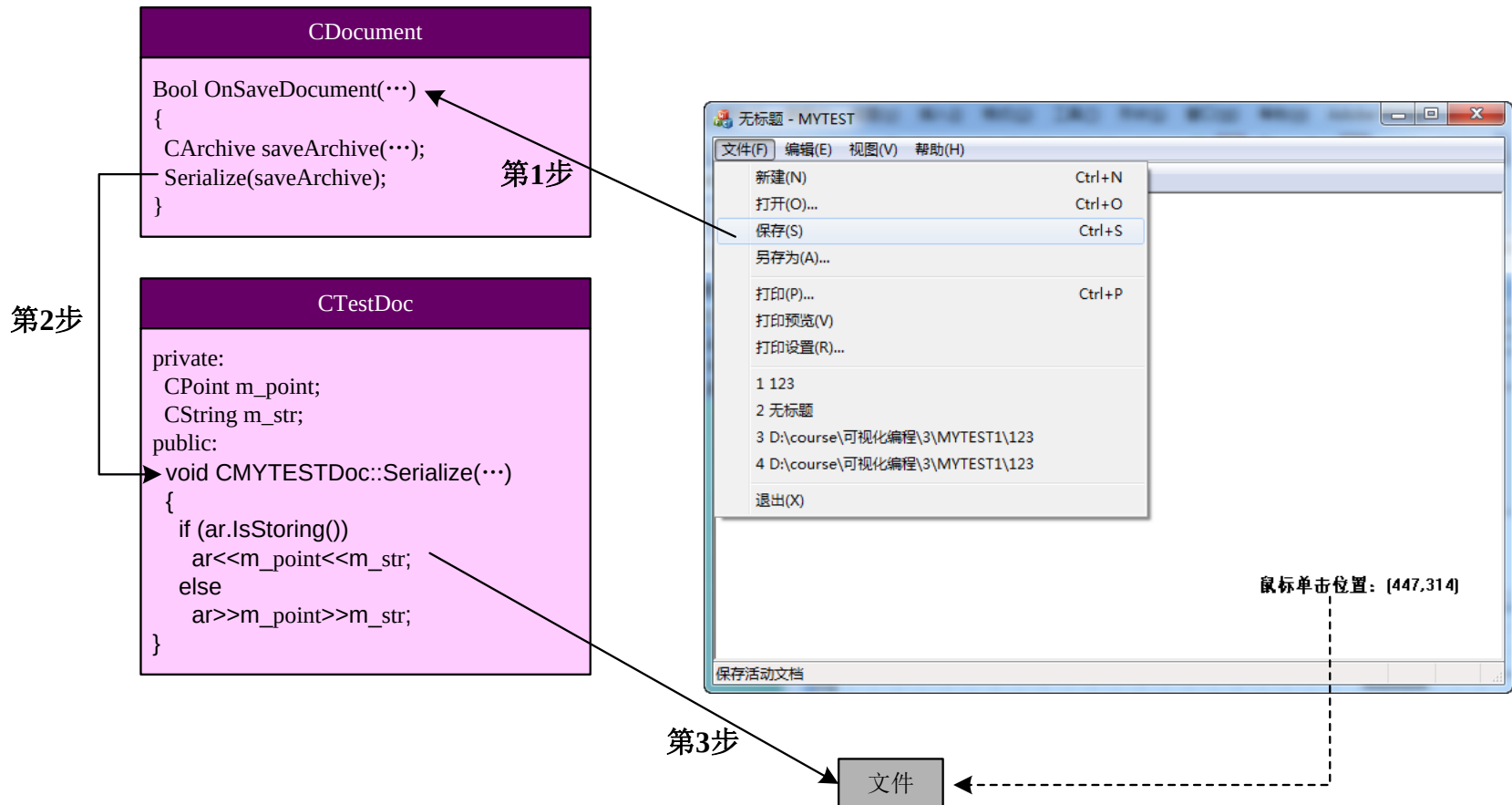
举例分析

➤ 鼠标点击过程



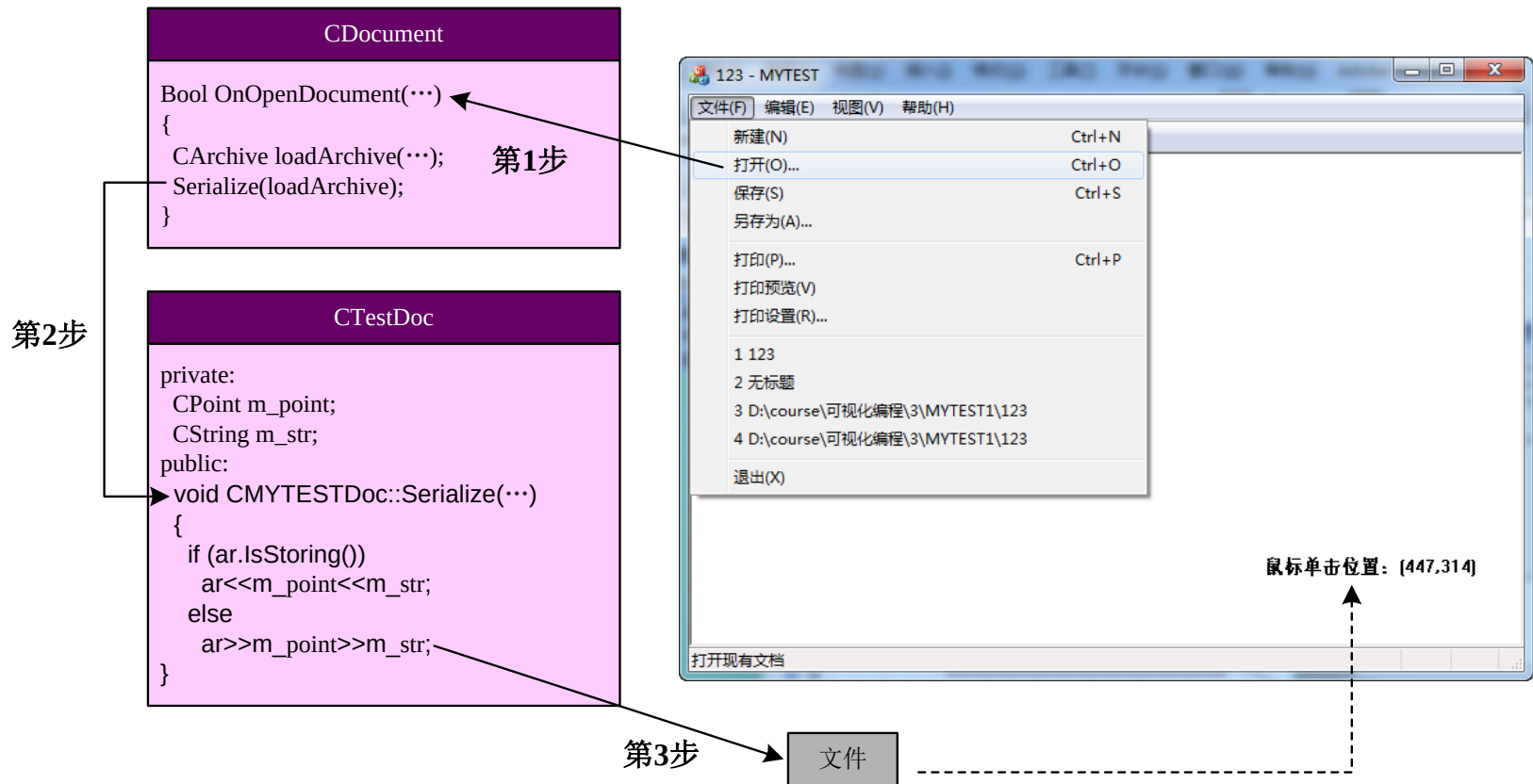
举例分析

➤ 文件保存过程



举例分析

➤ 文件读取过程



总结：文档/视图结构的优点

- 把数据处理类从用户界面处理类中分离出来，使得每一个类都能集中地执行一项工作
- 把Windows程序通常要做的工作分成若干定义好的类，有助于应用程序的模块化，程序易于扩展，编程时只需修改所涉及的类
- MFC应用程序框架起到了穿针引线的作用，按照消息处理函数功能的不同，将不同消息的响应分别分布在文档类和视图类中